

## Instituto Federal do Espírito Santo – Campus Serra

Curso de Engenharia de Controle e Automação

Disciplina: Robótica | Prof. Flávio Garcia Pereira

Grupo de Aprendizado de Máquina e Automação (GAMA)

# Trabalho Prático 1 – Parte 1

## Simulador de um Robô SCARA de 2 Graus de Liberdade

*Cinemática direta e inversa, geração de trajetória com limites de velocidade e aceleração, e execução a partir de arquivo de pontos*

### Resumo do trabalho

Você irá construir, em **Python**, um **simulador gráfico** de um robô SCARA planar de 2 GDL. A tela deve exibir o desenho do braço, que se movimenta assim que um comando é emitido. O simulador deverá: (i) implementar a **cinemática direta e inversa**; (ii) incorporar um **gerador de trajetória** que respeite limites de **velocidade** e **aceleração** das juntas; e (iii) **ler um arquivo de pontos** (todos dentro do espaço de trabalho) e executar o percurso animando o braço. Na **Parte 2** (semestre à frente), o mesmo simulador comandará o **robô SCARA real** – equipado com uma **ferramenta laser de gravação** – em espelhamento (inclusive em tempo real). Este roteiro já apresenta a modelagem, os requisitos e a análise de acionamento que preparam a integração física.

## Sumário

|     |                                                                           |   |
|-----|---------------------------------------------------------------------------|---|
| 1   | Contexto e objetivos                                                      | 2 |
| 2   | O robô e os parâmetros adotados                                           | 2 |
| 3   | Modelagem cinemática                                                      | 2 |
| 3.1 | Cinemática direta . . . . .                                               | 3 |
| 3.2 | Espaço de trabalho . . . . .                                              | 3 |
| 3.3 | Cinemática inversa . . . . .                                              | 4 |
| 3.4 | Validação numérica (obrigatória) . . . . .                                | 4 |
| 4   | Geração de trajetória com limites de velocidade e aceleração              | 5 |
| 4.1 | Perfil trapezoidal de velocidade . . . . .                                | 5 |
| 4.2 | Sincronização das juntas . . . . .                                        | 5 |
| 4.3 | Da cinemática à dinâmica: como escolher $v_{\max}$ e $a_{\max}$ . . . . . | 6 |
| 5   | Execução a partir de arquivo de pontos                                    | 7 |
| 5.1 | Formato do arquivo . . . . .                                              | 7 |
| 5.2 | Validação e execução . . . . .                                            | 7 |
| 6   | Requisitos do simulador (interface gráfica)                               | 7 |
| 7   | Extra (bônus): dinâmica e sugestão de controladores                       | 7 |
| 7.1 | Modelo dinâmico do 2R horizontal . . . . .                                | 8 |
| 7.2 | Estratégias de controle sugeridas . . . . .                               | 9 |

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| <b>8</b> | <b>Prévia da Parte 2: integração com o robô real</b>            | <b>9</b>  |
| 8.1      | Atuadores e drivers . . . . .                                   | 9         |
| 8.2      | Análise de resolução (e a recomendação de micropasso) . . . . . | 9         |
| 8.3      | Sincronismo, laser e segurança . . . . .                        | 10        |
| <b>9</b> | <b>Entregáveis, avaliação e cronograma</b>                      | <b>11</b> |
| <b>A</b> | <b>Sugestões de implementação em Python</b>                     | <b>11</b> |

## 1 Contexto e objetivos

O robô SCARA (*Selective Compliance Assembly Robot Arm*) é um dos manipuladores mais usados na indústria em tarefas de *pick-and-place*, montagem, dispensação e gravação, justamente por combinar rigidez vertical com boa mobilidade no plano horizontal. Nesta primeira parte trabalharemos com a versão didática de **2 GDL** do robô do laboratório: apenas as duas juntas rotativas do plano  $(\theta_1, \theta_2)$ , sem o eixo prismático vertical nem a rotação da ferramenta. Isso é suficiente para exercitar todo o núcleo de um controlador de robô: modelagem cinemática, planejamento de trajetória e limitação dinâmica.

### Objetivos de aprendizagem

1. Deduzir e implementar a **cinemática direta** (CD) e a **cinemática inversa** (CI) do manipulador 2R planar.
2. Determinar o **espaço de trabalho** e verificar a **alcançabilidade** de pontos.
3. Projetar um **gerador de trajetória** com **perfil trapezoidal** de velocidade, respeitando  $v_{\max}$  e  $a_{\max}$  de cada junta, com juntas **sincronizadas**.
4. Ler um **arquivo de pontos** e executar o percurso, **animando** o braço em tela.
5. Relacionar as escolhas de  $v_{\max}/a_{\max}$  com a **dinâmica** do robô e com o **acionamento** (motores de passo, drivers, resolução), preparando a Parte 2.

## 2 O robô e os parâmetros adotados

Trata-se de um manipulador **2R planar** operando no **plano horizontal** (visão de topo). A base é fixa; a primeira junta  $(\theta_1)$  gira o elo 1 (comprimento  $L_1$ ) em torno da origem; a segunda junta  $(\theta_2)$ , montada no cotovelo, gira o elo 2 (comprimento  $L_2$ ). A extremidade do elo 2 é o *ponto da ferramenta* (na Parte 2, o **cabeçote laser**).

Como valores de referência para o simulador e para todas as validações numéricas deste roteiro, adote:

$$L_1 = 0,200 \text{ m}, \quad L_2 = 0,150 \text{ m}.$$

Você pode ajustar esses comprimentos aos do robô real quando ele estiver disponível; a estrutura do código não deve depender de valores fixos (parametrize  $L_1, L_2$ ).

## 3 Modelagem cinemática

**Tabela 1:** Convenções e parâmetros do modelo.

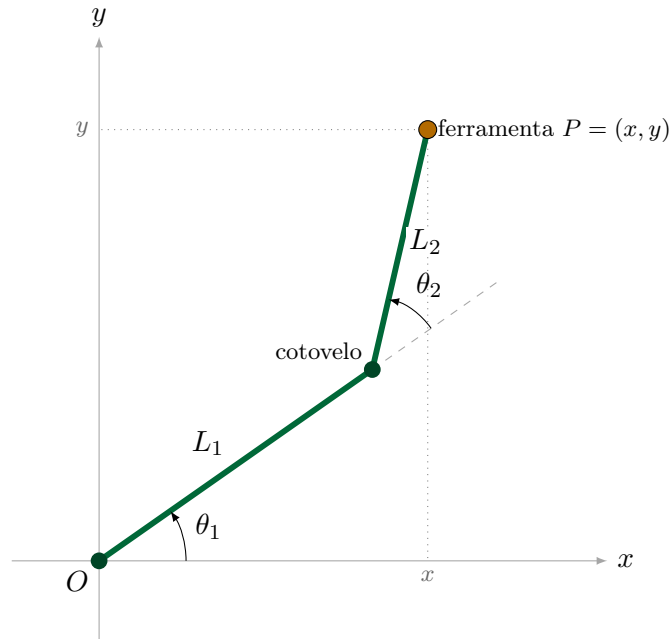
| Símbolo                        | Significado                             | Valor de referência |
|--------------------------------|-----------------------------------------|---------------------|
| $\theta_1, \theta_2$           | Ângulos das juntas (variáveis de junta) | —                   |
| $L_1, L_2$                     | Comprimentos dos elos                   | 0,200 m; 0,150 m    |
| $(x, y)$                       | Posição da ferramenta no plano          | m                   |
| $r = \sqrt{x^2 + y^2}$         | Distância radial da base à ferramenta   | m                   |
| $r_{\text{ext}} = L_1 + L_2$   | Raio externo do espaço de trabalho      | 0,350 m             |
| $r_{\text{int}} =  L_1 - L_2 $ | Raio interno do espaço de trabalho      | 0,050 m             |

### 3.1 Cinemática direta

Dadas as juntas  $(\theta_1, \theta_2)$ , a posição da ferramenta obtém-se somando as contribuições dos dois elos (Figura 1):

$$\begin{cases} x = L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2), \\ y = L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2). \end{cases} \quad (1)$$

Note que o ângulo absoluto do elo 2 é  $\theta_1 + \theta_2$ : como as juntas são *em série*, a orientação do segundo elo acumula a da primeira. A orientação da ferramenta no plano é  $\phi = \theta_1 + \theta_2$  (informação útil na Parte 2, para orientar o corte do laser, embora com 2 GDL não seja possível controlar  $\phi$  e  $(x, y)$  de forma independente).



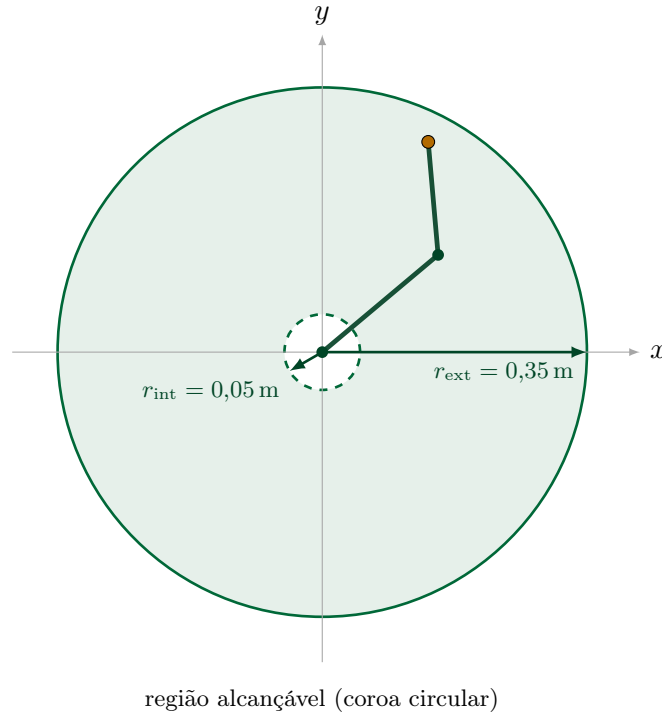
**Figura 1:** Manipulador 2R planar (SCARA de 2 GDL, visão de topo).  $\theta_1$  é medido do eixo  $x$  ao elo 1;  $\theta_2$ , da extensão do elo 1 ao elo 2.

### 3.2 Espaço de trabalho

Variando  $(\theta_1, \theta_2)$ , a ferramenta alcança um **anel** (coroa circular) centrado na base, de raio externo  $r_{\text{ext}} = L_1 + L_2 = 0,350$  m e raio interno  $r_{\text{int}} = |L_1 - L_2| = 0,050$  m (Figura 2). Um ponto  $(x, y)$  é **alcançável** se, e somente se,

$$|L_1 - L_2| \leq \sqrt{x^2 + y^2} \leq L_1 + L_2. \quad (2)$$

O raio interno existe porque, com  $L_1 \neq L_2$ , o braço não consegue “fechar” completamente sobre a base. Todo ponto lido do arquivo (Seção 5) deve ser testado por (2) antes de ser executado.



**Figura 2:** Espaço de trabalho do SCARA 2 GDL: coroa circular entre  $r_{\text{int}}$  e  $r_{\text{ext}}$ .

### 3.3 Cinemática inversa

Dado um alvo  $(x, y)$  alcançável, buscamos  $(\theta_1, \theta_2)$ . Usando a lei dos cossenos no triângulo  $O$ –cotovelo– $P$ :

$$\cos \theta_2 = \frac{x^2 + y^2 - L_1^2 - L_2^2}{2 L_1 L_2}, \quad \theta_2 = \pm \arccos(\cdot). \quad (3)$$

O sinal escolhe a configuração **cotovelo para baixo** ( $\theta_2 > 0$ ) ou **cotovelo para cima** ( $\theta_2 < 0$ ): há, em geral, **duas soluções** para o mesmo ponto (Figura 3). Em seguida,

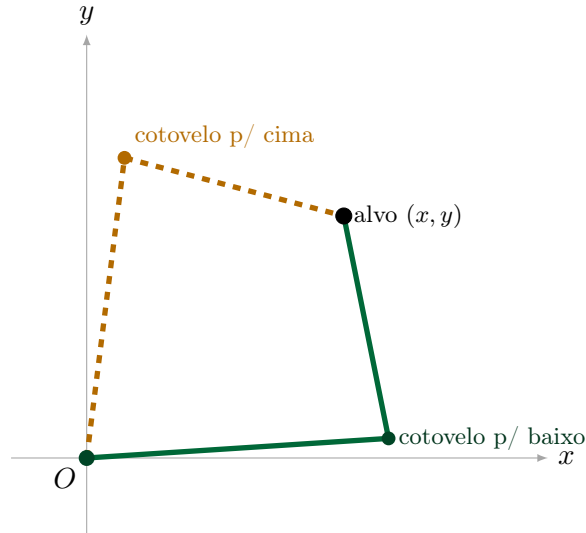
$$\theta_1 = \text{atan2}(y, x) - \text{atan2}(L_2 \sin \theta_2, L_1 + L_2 \cos \theta_2). \quad (4)$$

#### Cuidados numéricos na CI

- Use sempre `atan2` (nunca `atan`) para preservar o quadrante correto.
- Antes de `arccos`, verifique (2) e faça `clamp` do argumento em  $[-1, 1]$  para evitar NaN por erro de arredondamento.
- Nos limites do anel ( $r = r_{\text{ext}}$  ou  $r = r_{\text{int}}$ ) as duas soluções coincidem ( $\theta_2 = 0$  ou  $\theta_2 = \pm 180^\circ$ ) – **singularidades**. Trate-as sem quebrar o programa.
- Ofereça ao usuário a escolha do cotovelo e, na execução de trajetórias, mantenha a **continuidade** (não alterne de cotovelo entre pontos vizinhos).

### 3.4 Validação numérica (obrigatória)

Seu código deve reproduzir *exatamente* as tabelas abaixo (tolerância  $\leq 10^{-6}$ ). Elas foram pré-calculadas e verificadas; use-as como *testes automatizados*.



**Figura 3:** Duas soluções da cinemática inversa para o mesmo alvo: cotovelo para baixo (verde, contínuo) e para cima (âmbar, tracejado).

**Tabela 2:** Cinemática direta:  $(\theta_1, \theta_2) \rightarrow (x, y)$ , com  $L_1 = 0,20$  m,  $L_2 = 0,15$  m.

| $\theta_1$ (°) | $\theta_2$ (°) | $x$ (m) | $y$ (m) | $r$ (m) |
|----------------|----------------|---------|---------|---------|
| 0              | 0              | 0,3500  | 0,0000  | 0,3500  |
| 90             | 0              | 0,0000  | 0,3500  | 0,3500  |
| 0              | 90             | 0,2000  | 0,1500  | 0,2500  |
| 45             | 45             | 0,1414  | 0,2914  | 0,3239  |
| 30             | 60             | 0,1732  | 0,2500  | 0,3041  |
| 90             | 90             | -0,1500 | 0,2000  | 0,2500  |

## 4 Geração de trajetória com limites de velocidade e aceleração

Mover o braço “instantaneamente” de uma configuração a outra é fisicamente impossível e, no robô real, faria o motor **perder passos**. O simulador deve, portanto, gerar uma **trajetória temporal**  $\theta_i(t)$  que respeite, para cada junta, uma **velocidade máxima**  $v_{\max}$  e uma **aceleração máxima**  $a_{\max}$ .

### 4.1 Perfil trapezoidal de velocidade

A abordagem clássica é o **perfil trapezoidal**: acelera com  $a_{\max}$ , cruza com  $v_{\max}$  constante e desacelera com  $-a_{\max}$ . Para um deslocamento  $\Delta\theta = |\theta_{\text{fim}} - \theta_{\text{ini}}|$ :

$$t_a = \frac{v_{\max}}{a_{\max}} \text{ (rampa)}, \quad \Delta\theta_a = \frac{1}{2}a_{\max}t_a^2 \text{ (por rampa)}. \quad (5)$$

- Se  $2\Delta\theta_a < \Delta\theta$ : perfil **trapezoidal**, com fase de cruzeiro de duração  $t_c = \frac{\Delta\theta - 2\Delta\theta_a}{v_{\max}}$  e tempo total  $T = 2t_a + t_c$ .
- Caso contrário: perfil **triangular** (não se atinge  $v_{\max}$ ), com  $t_a = \sqrt{\Delta\theta/a_{\max}}$ ,  $t_c = 0$  e  $T = 2t_a$ .

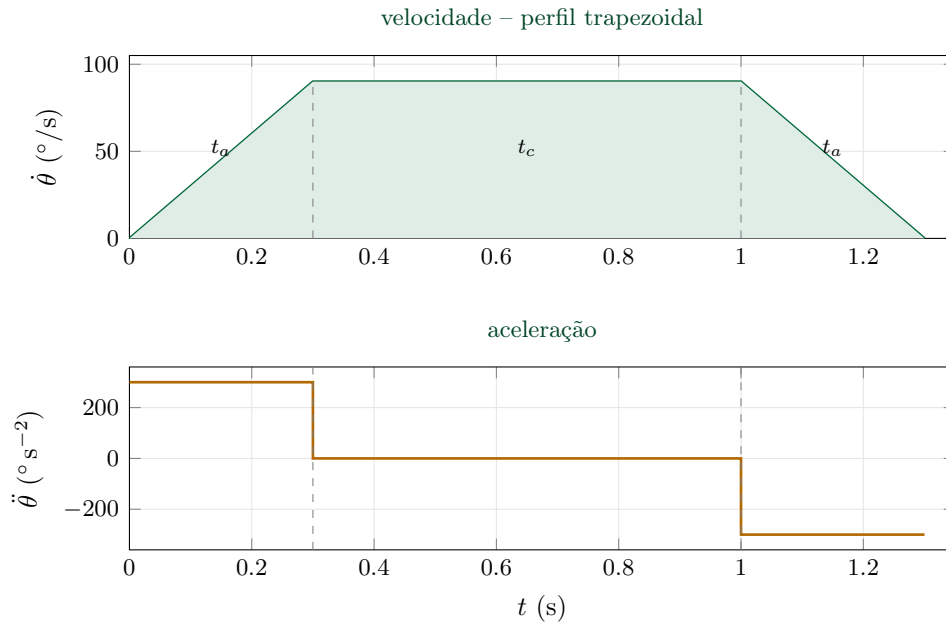
A Figura 4 ilustra o caso trapezoidal ( $\Delta\theta = 90^\circ$ ,  $v_{\max} = 90^\circ/\text{s}$ ,  $a_{\max} = 300^\circ/\text{s}^2$ :  $t_a = 0,30$  s,  $t_c = 0,70$  s,  $T = 1,30$  s).

### 4.2 Sincronização das juntas

Ao ir de uma pose a outra, as duas juntas têm deslocamentos diferentes e, portanto, tempos  $T_1 \neq T_2$ . Para que a ferramenta chegue com as duas juntas **ao mesmo tempo** (movimento

**Tabela 3:** Cinemática inversa:  $(x, y) \rightarrow (\theta_1, \theta_2)$ , para as duas configurações de cotovelo.

| Alvo (m) |        | Cotovelo p/ baixo ( $^\circ$ ) |            | Cotovelo p/ cima ( $^\circ$ ) |            |
|----------|--------|--------------------------------|------------|-------------------------------|------------|
| $x$      | $y$    | $\theta_1$                     | $\theta_2$ | $\theta_1$                    | $\theta_2$ |
| 0,2000   | 0,1500 | 0,00                           | 90,00      | 73,74                         | -90,00     |
| 0,1414   | 0,2914 | 45,00                          | 45,00      | 83,23                         | -45,00     |
| 0,1732   | 0,2500 | 30,00                          | 60,00      | 80,57                         | -60,00     |
| -0,1500  | 0,2000 | 90,00                          | 90,00      | 163,74                        | -90,00     |



**Figura 4:** Perfil trapezoidal de velocidade e sua aceleração (constante por trechos). A área sob a curva de velocidade é o deslocamento  $\Delta\theta$ .

coordenado), adote o **tempo comum**  $T = \max(T_1, T_2)$  e re-escalone a junta mais rápida para durar  $T$  (reduzindo seu  $v$  e/ou  $a$  efetivos). Assim nenhuma junta ultrapassa seus limites e o movimento é suave.

#### Módulos exigidos

Implemente a limitação como **módulos separados e reutilizáveis**: (1) um gerador de perfil por junta (entrada:  $\Delta\theta, v_{\max}, a_{\max}$ ; saída:  $\theta(t), \dot{\theta}(t)$ ); (2) um sincronizador que impõe  $T$  comum; (3) um laço de animação que amostra  $\theta_i(t)$  no passo de tempo do simulador (ex. 20 ms, 50 fps) e redesenha o braço.

### 4.3 Da cinemática à dinâmica: como escolher $v_{\max}$ e $a_{\max}$

No simulador,  $v_{\max}$  e  $a_{\max}$  são parâmetros de projeto. No robô real, eles não são arbitrários: decorrem do **torque disponível** nos motores e da **inércia** do braço. Modelando o 2R no plano *horizontal* (sem gravidade – ver Seção 7), o torque na junta 1 satisfaz  $\tau_1 \approx M_{11}(\theta_2) \ddot{\theta}_1 + (\text{acoplamento})$ , com a inércia efetiva máxima quando o braço está **estendido** ( $\theta_2 = 0$ ). Para os valores de referência e elos modelados como barras uniformes ( $m_1 = 0,6 \text{ kg}$ ,  $m_2 = 0,4 \text{ kg}$ ):

$$M_{11}^{\max} \approx 0,039 \text{ kg m}^2 \Rightarrow a_{\max} \lesssim \frac{\tau_{\text{disp}}}{M_{11}^{\max}}.$$

**Tabela 4:** Aceleração máxima admissível na junta 1 em função do torque disponível (estimativa, braço estendido).

| $\tau_{\text{disp}}$ (N m) | $a_{\text{max}}$ (rad s <sup>-2</sup> ) | $a_{\text{max}}$ (° s <sup>-2</sup> ) |
|----------------------------|-----------------------------------------|---------------------------------------|
| 0,8                        | 20,5                                    | 1.175                                 |
| 1,2                        | 30,8                                    | 1.763                                 |
| 2,0                        | 51,3                                    | 2.938                                 |

Ou seja: valores didáticos como  $a_{\text{max}} = 300^\circ \text{s}^{-2}$  e  $v_{\text{max}} = 90^\circ/\text{s}$  to  $180^\circ/\text{s}$  ficam confortavelmente dentro da capacidade dos motores previstos (Seção 8) – e a checagem de frequência de pulsos (Tabela 8) confirma que o *driver* não é o gargalo. O objetivo pedagógico é que você **justifique** os limites escolhidos, e não apenas os chute.

## 5 Execução a partir de arquivo de pontos

O simulador deve carregar um **arquivo de texto** (formato CSV) com uma sequência de pontos e executar o percurso, movimentando o braço ponto a ponto com os perfis limitados.

### 5.1 Formato do arquivo

Uma linha por ponto. Sugestão de formato (com cabeçalho e comentários iniciados por #):

```

1 # pontos.csv -- coordenadas da ferramenta (metros), cotovelo: 0=baixo 1=cima
2 # x, y, cotovelo
3 0.30, 0.00, 0
4 0.25, 0.10, 0
5 0.20, 0.15, 0
6 0.10, 0.25, 0
7 0.00, 0.30, 0

```

Opcionalmente, aceite também linhas em **espaço de juntas** ( $\theta_1, \theta_2$ ) e um campo de estado da ferramenta (ex. laser ligado/desligado), já pensando na Parte 2 (gravação).

### 5.2 Validação e execução

Para cada ponto: (1) verifique a alcançabilidade por (2) – **rejeite** e sinalize pontos fora do espaço, sem travar o programa; (2) calcule a CI mantendo continuidade de cotovelo; (3) gere a trajetória sincronizada do ponto atual ao próximo; (4) anime. Ao final, o braço deve ter percorrido toda a lista. Exiba, se possível, o **rastro** da ferramenta na tela.

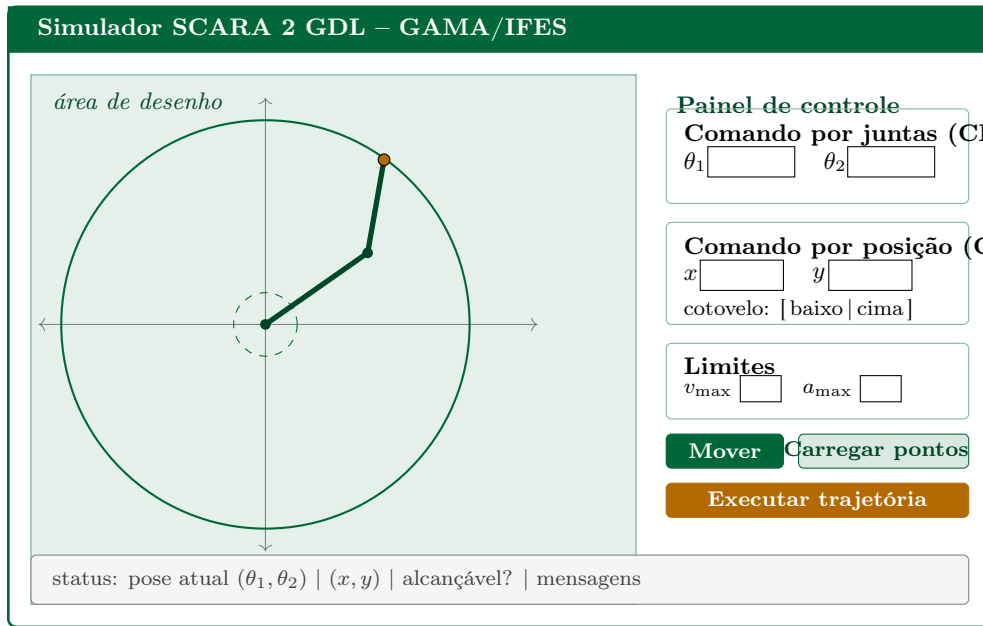
## 6 Requisitos do simulador (interface gráfica)

A janela (Figura 5) deve conter uma **área de desenho** com o braço, os eixos e o contorno do espaço de trabalho, e um **painel de controle** com: campos para  $\theta_1, \theta_2$  (comando por CD) e para  $x, y$  (comando por CI, com seleção de cotovelo); botões *Mover*, *Carregar pontos* e *Executar*; campos de  $v_{\text{max}}, a_{\text{max}}$ ; e uma **barra de status** (pose atual, alcançabilidade, mensagens). O braço deve **mover-se assim que o comando for emitido**, de forma animada (não “teletransportar”).

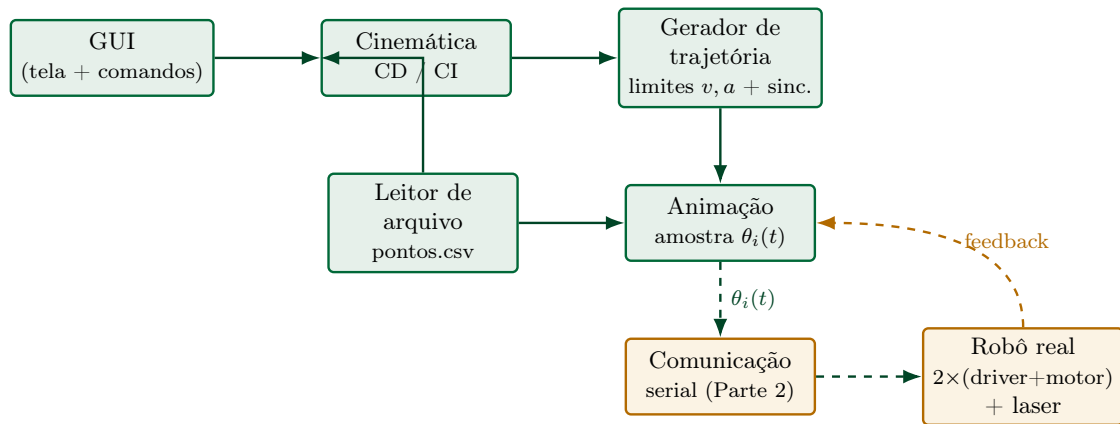
A Figura 6 mostra a **arquitetura de software** sugerida. Módulos bem separados facilitam a Parte 2: basta acoplar o bloco de comunicação sem reescrever a cinemática.

## 7 Extra (bônus): dinâmica e sugestão de controladores

Esta seção é **opcional** e vale pontuação extra. Ela conecta o simulador cinemático a um modelo dinâmico – exatamente a ponte para a disciplina de Controle.



**Figura 5:** Esboço (mock-up) da interface: área de desenho à esquerda; painel de comandos à direita; barra de status na base.



**Figura 6:** Arquitetura de software. Verde: Parte 1 (simulador). Âmbar/tracejado: Parte 2 (integração com o robô real).

## 7.1 Modelo dinâmico do 2R horizontal

Como o robô opera no **plano horizontal**, **não há termo gravitacional** atuando sobre as juntas – uma simplificação elegante em relação ao pêndulo duplo vertical. A equação de movimento fica

$$M(\mathbf{q}) \ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}} = \boldsymbol{\tau}, \quad \mathbf{q} = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}, \quad (6)$$

com matriz de inércia

$$M(\mathbf{q}) = \begin{bmatrix} I_1 + I_2 + m_1 l_{c1}^2 + m_2 (L_1^2 + l_{c2}^2 + 2L_1 l_{c2} \cos \theta_2) & I_2 + m_2 (l_{c2}^2 + L_1 l_{c2} \cos \theta_2) \\ I_2 + m_2 (l_{c2}^2 + L_1 l_{c2} \cos \theta_2) & I_2 + m_2 l_{c2}^2 \end{bmatrix}, \quad (7)$$

e termos de Coriolis/centrífugos com  $h = m_2 L_1 l_{c2} \sin \theta_2$ :

$$C(\mathbf{q}, \dot{\mathbf{q}}) = \begin{bmatrix} -h \dot{\theta}_2 & -h(\dot{\theta}_1 + \dot{\theta}_2) \\ h \dot{\theta}_1 & 0 \end{bmatrix}. \quad (8)$$

Aqui  $l_{ci}$  é a distância ao centro de massa e  $I_i$  a inércia própria de cada elo. Perceba o **acoplamento**: a inércia vista por  $\theta_1$  depende de  $\theta_2$  (máxima com o braço estendido, como usamos na Seção 5.3).



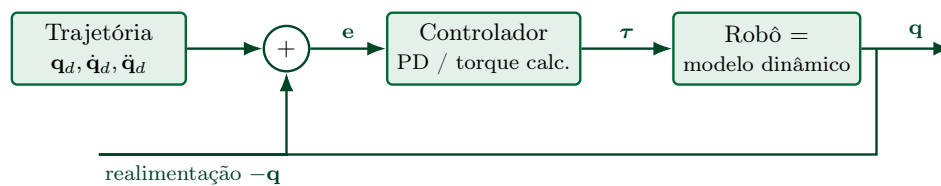
## 7.2 Estratégias de controle sugeridas

Três níveis, em ordem crescente de sofisticação (Figura 7):

1. **Cinemático / malha aberta** – é o que o robô real fará: o motor de passo segue a sequência de pulsos gerada pela trajetória, *sem realimentação*. O “controle” é o planejamento (perfil trapezoidal) que garante que nenhum passo seja perdido. É o mínimo exigido.
2. **PD no espaço de juntas** – sobre o modelo (6),  $\tau = K_p \mathbf{e} + K_d \dot{\mathbf{e}}$ , com  $\mathbf{e} = \mathbf{q}_d - \mathbf{q}$ . Simples e didático para observar erro de rastreamento e efeito do acoplamento.
3. **Torque calculado (*computed torque*)** – linearização por realimentação:

$$\tau = M(\mathbf{q})(\ddot{\mathbf{q}}_d + K_d \dot{\mathbf{e}} + K_p \mathbf{e}) + C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}, \quad (9)$$

que cancela a dinâmica não linear e impõe erro com dinâmica  $\ddot{\mathbf{e}} + K_d \dot{\mathbf{e}} + K_p \mathbf{e} = 0$ . Excelente para demonstrar rastreamento de trajetória de alta qualidade.



**Figura 7:** Malha de controle no espaço de juntas: a trajetória gerada (com limites  $v, a$ ) é a referência do controlador.

## 8 Prévia da Parte 2: integração com o robô real

Na Parte 2, o simulador comandará o **SCARA físico**, que se moverá **em espelhamento** com a tela (inclusive simultaneamente). A ferramenta é um **laser de gravação**. Já apresentamos aqui o acionamento para que você projete o simulador com a resolução correta desde o início.

### 8.1 Atuadores e drivers

**Tabela 5:** Cadeia de acionamento prevista.

|                      | Junta $\theta_1$     | Junta $\theta_2$         |
|----------------------|----------------------|--------------------------|
| Motor                | NEMA 23 (1,8°/passo) | NEMA 17 (1,8°/passo)     |
| Redução              | <b>direta</b> (1:1)  | <b>planetária 5,18:1</b> |
| Driver               | DM556                | DM556 (ou DRV8825/TMC)   |
| Passos/volta (motor) | 200                  | 200                      |

O DM556 aceita micropasso de 200 a 25.600 pulsos/volta (via DIP) e frequência de pulso de até 200 kHz – folga enorme para as velocidades deste robô.

### 8.2 Análise de resolução (e a recomendação de micropasso)

**Junta  $\theta_1$  (sem redução).** Como o motor aciona o elo diretamente, a resolução da junta é a resolução do micropasso. E, por mover *todo* o braço, o erro cartesiano é o pior ( $r$  até 0,35 m):

**Tabela 6:** Resolução de  $\theta_1$  (NEMA 23 direto, DM556). Cartesiano avaliado em  $r = 0,35$  m.

| Micropasso  | Pulsos/volta | °/passo | Cartesiano (mm) |
|-------------|--------------|---------|-----------------|
| 1/4         | 800          | 0,4500  | 2,7489          |
| 1/8 (atual) | 1.600        | 0,2250  | 1,3744          |
| 1/16        | 3.200        | 0,1125  | 0,6872          |
| 1/32        | 6.400        | 0,0563  | 0,3436          |
| 1/64        | 12.800       | 0,0281  | 0,1718          |
| 1/128       | 25.600       | 0,0141  | 0,0859          |

### Resposta à pergunta “1/8 ou aumentar a resolução?”

**Aumente.** Em 1/8 a resolução cartesiana no alcance máximo é de  $\approx 1,37$  mm – grosseira para gravação a laser. Recomendo **1/16** (3.200 pulsos/volta,  $\approx 0,69$  mm) como bom compromisso; **1/32** ( $\approx 0,34$  mm) se quiser movimento mais suave e fino. *Ressalva importante:* micropasso melhora a granularidade do comando e a suavidade, mas **não** melhora proporcionalmente a exatidão estática (há erro de micropasso e queda de torque incremental). O verdadeiro gargalo de  $\theta_1$  é a **ausência de redução**: se a aplicação exigir exatidão  $\lesssim 0,1$  mm, o ideal é acrescentar uma **redução** (correia/planetária) em  $\theta_1$ , e não apenas subir o micropasso.

**Junta  $\theta_2$  (redução 5,18:1).** A redução *multiplica* a resolução: mesmo em passo cheio, a resolução de saída ( $\approx 0,35^\circ$ ) já é fina, e como  $\theta_2$  move só o elo 2 (braço de alavanca  $L_2 = 0,15$  m), o erro cartesiano é pequeno.

**Tabela 7:** Resolução de  $\theta_2$  (NEMA 17 com redução 5,18:1). Cartesiano avaliado em  $L_2 = 0,15$  m.

| Micropasso | Pulsos/volta (saída) | °/passo (saída) | Cartesiano (mm) |
|------------|----------------------|-----------------|-----------------|
| 1/1        | 1.036,0              | 0,3475          | 0,9097          |
| 1/2        | 2.072,0              | 0,1737          | 0,4549          |
| 1/4        | 4.144,0              | 0,0869          | 0,2274          |
| 1/8        | 8.288,0              | 0,0434          | 0,1137          |
| 1/16       | 16.576,0             | 0,0217          | 0,0569          |

### Recomendação para $\theta_2$

Use **1/8** (resolução cartesiana  $\approx 0,11$  mm). Isso deixa  $\theta_2$  *mais fino* do que  $\theta_1$ , garantindo que  $\theta_1$  seja o fator limitante (o que é desejável). *Balanceamento fino:* como o braço de alavanca de  $\theta_1$  ( $\approx 0,35$  m) é cerca de  $2,3\times$  o de  $\theta_2$  (0,15 m), para *igualar* a resolução cartesiana bastaria  $\theta_2$  em  $\approx 1/2$ . Manter 1/8 dá suavidade com folga e pulsos ainda modestos.

**Tabela 8:** Checagem de frequência de pulsos na escolha recomendada (driver suporta 200 kHz).

| Eixo / micropasso | $v_{\max}$ (°/s) | Freq. de pulso (kHz) |
|-------------------|------------------|----------------------|
| $\theta_1$ @ 1/16 | 180              | 1,60                 |
| $\theta_1$ @ 1/32 | 180              | 3,20                 |
| $\theta_2$ @ 1/8  | 120              | 2,76                 |

## 8.3 Sincronismo, laser e segurança

O simulador enviará a sequência de posições/pulsos ao robô por porta serial (ex. Arduino/ESP32 gerando os pulsos PUL/DIR). Para espelhamento em tempo real, use um **relógio comum** e

envie *setpoints* nas mesmas amostras usadas na animação. O acionamento do laser (ligar/desligar, potência) deve ser um **canal separado**, sincronizado com o movimento e com **intertravamento de segurança** (o laser só liga com o braço em movimento comandado e dentro do espaço de trabalho). Detalhes completos virão no roteiro da Parte 2.

## 9 Entregáveis, avaliação e cronograma

### Entregáveis

1. **Código-fonte** em Python (organizado em módulos: cinemática, trajetória, GUI, leitor de arquivo), com instruções de execução.
2. **Demonstração** do simulador: comando por CD, por CI (com escolha de cotovelo), rejeição de ponto fora do espaço, e execução completa de um **pontos.csv**.
3. **Relatório** curto (máx. 8 páginas) com: dedução da CD/CI, tabelas de validação reproduzidas pelo seu código, descrição do gerador de trajetória, e justificativa dos limites  $v_{\max}/a_{\max}$ .
4. (Bônus) Modelo dinâmico e/ou controlador PD ou torque calculado.

**Tabela 9:** Critérios de avaliação.

| Item                                                                                | Pontos     |
|-------------------------------------------------------------------------------------|------------|
| Cinemática direta e inversa corretas (batem com as tabelas de validação)            | 25         |
| Interface gráfica com braço animado que se move ao comando                          | 20         |
| Gerador de trajetória com limites de $v$ e $a$ (perfil trapezoidal + sincronização) | 25         |
| Leitura de arquivo, validação de espaço de trabalho e execução do percurso          | 20         |
| Organização do código, relatório e justificativa dos limites                        | 10         |
| <b>Total</b>                                                                        | <b>100</b> |
| Bônus: dinâmica / controlador (PD ou torque calculado)                              | +15        |

**Grupos:** 2 a 3 integrantes. **Prazo sugerido:** 3 semanas (S1: cinemática + tela; S2: trajetória + limites; S3: arquivo + integração e relatório).

## A Sugestões de implementação em Python

**Bibliotecas.** Para a GUI/animação, qualquer uma serve: **tkinter**+**matplotlib** (mais simples), **pygame** (mais fluido) ou **PyQt**. Use **numpy** para a matemática. Estruture o código em módulos independentes – isso é o que permitirá plugar o robô real na Parte 2 sem reescrever a cinemática.

**Núcleo cinemático (esqueleto):**

```

1 import numpy as np
2
3 L1, L2 = 0.20, 0.15 # metros (parametrize!)
4
5 def forward(t1, t2):
6     """Cinemática direta: juntas (rad) -> (x, y)."""
7     x = L1*np.cos(t1) + L2*np.cos(t1 + t2)
8     y = L1*np.sin(t1) + L2*np.sin(t1 + t2)
9     return x, y
10

```

```
11 def reachable(x, y, tol=1e-9):
12     r = np.hypot(x, y)
13     return abs(L1 - L2) - tol <= r <= (L1 + L2) + tol
14
15 def inverse(x, y, elbow="down"):
16     """Cinematica inversa: (x, y) -> (t1, t2) em rad, ou None se fora."""
17     if not reachable(x, y):
18         return None
19     c2 = (x*x + y*y - L1**2 - L2**2) / (2*L1*L2)
20     c2 = np.clip(c2, -1.0, 1.0) # evita NaN por arredondamento
21     s2 = np.sqrt(max(0.0, 1 - c2*c2))
22     if elbow == "up":
23         s2 = -s2
24     t2 = np.arctan2(s2, c2)
25     t1 = np.arctan2(y, x) - np.arctan2(L2*np.sin(t2), L1 + L2*np.cos(t2))
26     return t1, t2
```

### Perfil trapezoidal (uma junta):

```
1 def trapezoidal(dq, vmax, amax):
2     """Retorna funcao theta(t) e duracao T para deslocamento dq (rad)."""
3     s = np.sign(dq); dq = abs(dq)
4     ta = vmax / amax
5     da = 0.5 * amax * ta**2
6     if 2*da >= dq: # perfil triangular
7         ta = np.sqrt(dq / amax); tc = 0.0; vpk = amax*ta
8     else: # perfil trapezoidal
9         vpk = vmax; tc = (dq - 2*da) / vmax
10    T = 2*ta + tc
11    def theta(t):
12        t = np.clip(t, 0, T)
13        if t < ta:
14            q = 0.5*amax*t**2
15        elif t < ta + tc:
16            q = da + vpk*(t - ta)
17        else:
18            td = t - ta - tc
19            q = da + vpk*tc + vpk*td - 0.5*amax*td**2
20    return s*q
21    return theta, T
```

Para sincronizar as duas juntas em um movimento, gere os perfis de cada uma, tome  $T = \max(T_1, T_2)$  e re-escalone a mais rápida (amostrando ambas em  $t \in [0, T]$ ).

*Bom trabalho! Guardem o código bem modularizado – ele é a base direta da Parte 2.*

## Referências

- [1] CRAIG, J. J. *Introduction to Robotics: Mechanics and Control*. 4. ed. Pearson, 2018.
- [2] SPONG, M. W.; HUTCHINSON, S.; VIDYASAGAR, M. *Robot Modeling and Control*. 2. ed. Wiley, 2020.
- [3] SICILIANO, B. *et al. Robotics: Modelling, Planning and Control*. Springer, 2009.
- [4] LYNCH, K. M.; PARK, F. C. *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017.
- [5] LEADSHINE. *DM556 Digital Stepper Drive User Manual*, Rev. 3.0, 2019.